

Chapter 3: AI-Driven Portable On-Premise IoT Cloud Mini Server for Secure Sensor Data Management in Future Classrooms and MSME Applications

Ithaya Priya B^{1*}, Megna K¹, Lavanya S¹, Venkatesan R²

¹Department of Computer Science and Engineering, E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu, India

²Department of Biomedical Engineering, E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu, India

*Corresponding Author Email: ithayapriya@egspec.org

Abstract

The rapid adoption of Internet of Things (IoT) technologies across academic institutions and Micro, Small, and Medium Enterprises (MSMEs) has created a growing need for affordable, secure, and locally managed data infrastructure. Today, most IoT deployments depend on centralized third-party cloud platforms such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud. While these services offer scalability, they come with significant recurring costs, require uninterrupted internet access, and raise serious concerns about data privacy and institutional ownership challenges that directly affect undergraduate engineering labs, rural innovation centers, and small businesses. It presents the design and development of a Portable On-Premise IoT Cloud Mini Server — a compact, energy-efficient hardware–software system that replicates cloud-like capabilities entirely within a local network. Built on a Raspberry Pi 5 running Ubuntu Server 24.04 LTS, the system supports real-time sensor data acquisition, encrypted storage, time-series analysis, and AI-ready data pipelines all without depending on external internet connectivity. The security architecture incorporates AES-256 encryption, Argon2 password hashing, JSON Web Token (JWT) authentication, and a reverse proxy firewall. Designed for future classrooms and MSME environments, the system supports hands-on IoT experimentation, AI integration at the edge level, and full institutional control over sensitive sensor data. This work demonstrates how a portable, cost-effective, and self-hosted IoT infrastructure can serve as a foundation for AI-driven learning, real-time monitoring, and data sovereignty in resource-constrained environments.

Keywords: IoT Cloud Server, On-Premise Infrastructure, Raspberry Pi 5, Edge Computing, Data Sovereignty, AI-Driven Classroom, MSME IoT, Real-Time Data Streaming, AES-256 Encryption, Future Classrooms

1. Introduction

The Internet of Things (IoT) has quietly revolutionized the way we interact with the physical world. From smart home devices to industrial automation systems, IoT connects physical sensors and actuators to digital networks, enabling real-time data collection, monitoring, and intelligent decision-making [1]. Over the past decade, this technology has found its way into classrooms and small businesses, opening new possibilities for experiential learning, process optimization, and low-cost automation [3].

For undergraduate engineering students, IoT offers a hands-on gateway into the world of embedded systems, networking, and data analytics. For MSMEs, it provides an affordable means to modernize operations and monitor critical parameters such as temperature, humidity, energy consumption, and machine performance. However, despite these opportunities, a critical infrastructure gap exists: the majority of IoT platforms available today are designed for large organizations with steady internet access and substantial budgets making them poorly suited for academic labs and small enterprises [4].

Traditional cloud-based IoT solutions, while powerful, carry several limitations when deployed in educational or small-business settings. They require continuous internet connectivity, which may be unreliable in rural campuses. Monthly subscription charges accumulate over time, making large-scale student projects financially unsustainable. Sensitive sensor data is stored on third-party servers, creating data ownership concerns [6]. The backend infrastructure is largely inaccessible to students, limiting learning opportunities at the system level.

These challenges have sparked interest in on-premise solutions locally deployed servers that offer cloud-like capabilities without relying on external platforms [8]. The concept of a Portable IoT Cloud Mini Server addresses exactly this need. It brings the computational power of cloud infrastructure into a compact, energy-efficient device that can be placed in a classroom, laboratory, or MSME workshop, operated independently of the internet, and customized freely by its users [11].

This chapter presents the complete design, development, and analysis of such a system a Raspberry Pi 5-based portable mini server that functions as a local IoT cloud, capable of securely collecting sensor data from multiple devices, managing user projects, storing time-series records, and streaming live data to a web dashboard [14]. Crucially, the system is architected to be AI-ready, meaning it structures and stores data in formats that can feed directly into machine learning models for predictive analytics, anomaly detection, and smart classroom automation [7]. This chapter presents the design and development of a portable, AI-enabled on-premise IoT cloud mini server that bridges the gap between enterprise-grade cloud technology and real-world educational and MSME deployment [8].

2. Background and Existing Systems

To understand the significance of a portable on-premise IoT solution, it is important to first examine the current landscape of IoT cloud platforms and their limitations in educational and MSME contexts.

2.1 Public Cloud IoT Platforms

Major cloud providers have developed specialized IoT services that dominate the market. AWS IoT Core allows devices to communicate securely with cloud applications. Microsoft Azure IoT Hub provides enterprise-grade device management and analytics [13]. Google Cloud IoT Core (now succeeded by partner solutions) offered scalable data ingestion and AI integration. These platforms are feature-rich and battle-tested for large-scale enterprise deployments. However, they are not designed with academic or MSME users in mind [18].

The primary drawbacks observed in academic and MSME contexts include: high recurring costs based on data volume and API calls, mandatory internet connectivity for all operations, limited backend access that prevents infrastructure-level learning, data stored on remote servers governed by third-party terms of service, and complex onboarding procedures that overwhelm beginner students [9].

2.2 Edge Computing and Local IoT Alternatives

Edge computing has emerged as a promising paradigm that brings data processing closer to the source of data generation. Unlike cloud computing, where data travels to remote data centers, edge computing processes data locally on the device or on a nearby server. This reduces latency, conserves bandwidth, and enables offline operation [10].

Several open-source platforms have attempted to provide local IoT hosting capabilities. Tools like Grafana, InfluxDB, Node-RED, and Home Assistant offer components of a local IoT stack [11]. However, they lack integrated security hardening, structured project management for multiple users, and a cohesive product-oriented design suitable for academic or institutional deployment. No existing compact, ready-to-deploy solution specifically targets the academic laboratory and MSME ecosystem with a focus on portability, AI readiness, and institutional data ownership [20].

Table 1: Comparison of IoT Infrastructure Approaches

Feature	Public Cloud (AWS/Azure)	Basic Local Server	Proposed Mini Cloud Server
Internet Dependency	High	Low	None
Data Sovereignty	Limited	Moderate	Full
Cost (Recurring)	High	Low	Very Low
Portability	N/A	Poor	Excellent
Real-Time Streaming	Yes	Limited	Yes (WebSocket)
AI Readiness	Yes	No	Yes (On-Premise)
Security Level	High	Basic	High (AES-256)
Academic Customization	Limited	Moderate	Full

Table 1 highlights the key advantages of the proposed system over both public cloud platforms and basic local server setups, particularly in terms of data sovereignty, portability, and AI readiness.

3. Problem Statement

Despite the proven benefits of IoT technology in academic and industrial settings, the following critical challenges remain unaddressed for educational institutions and MSMEs:

- **Dependence on Third-Party Cloud Platforms:** Institutions and small businesses lack direct control over the infrastructure that stores and processes their sensor data. This creates both financial dependency and data sovereignty concerns.
- **Internet Dependency:** Cloud-based IoT systems fail or become inaccessible during network outages, which are common in rural campuses and small industrial units.
- **High Operational Costs:** Subscription-based billing models make it financially unsustainable for students running multiple simultaneous projects or for MSMEs with limited IT budgets.
- **Data Privacy Risks:** Sensitive experimental or business data transmitted to remote servers may be subject to jurisdictional regulations, vendor lock-in, and unauthorized access.
- **Absence of AI Integration at Local Level:** Most local IoT tools do not provide structured data pipelines that feed directly into AI or machine learning workflows, making on-premise AI deployment difficult.

The core problem is the absence of a compact, secure, portable, and affordable IoT cloud infrastructure that educational institutions and MSMEs can deploy, own, and operate independently — while also being ready to support AI-driven analytics and automation.

4. Objectives of the Study

The primary goal of this work is to design and develop a Portable On-Premise IoT Cloud Mini Server that addresses the above challenges. The specific objectives are:

- a. To develop a portable IoT mini server for local, internet-independent operation.
- b. To implement secure data collection using authentication, encryption, and access control.
- c. To ensure complete local storage and control of IoT sensor data.
- d. To support automatic registration of new sensor parameters.
- e. To create an AI-ready platform for time-series data analysis.
- f. To evaluate the system's performance, cost-effectiveness, and educational applications.

5. Proposed System Overview

The proposed Portable On-Premise IoT Cloud Mini Server is designed as a practical and user-friendly solution that brings advanced IoT cloud capabilities into a compact, low-power device. Unlike traditional large-scale servers, this system is small enough to be placed on a laboratory bench or within a small industrial workspace, making it highly suitable for undergraduate laboratories and MSME environments. The core idea behind this system is simple yet impactful: instead of relying on distant cloud infrastructures, it enables users to create their own localized cloud environment within their premises.

In conventional IoT setups, sensor data generated by devices is typically transmitted to remote servers managed by major technology providers. While this approach offers scalability, it also introduces challenges such as dependency on internet connectivity, recurring subscription costs, and concerns regarding data privacy. The proposed system takes a different approach by functioning as a private cloud that operates entirely within the institution's local network. This means that all data generated by IoT devices remains within the physical boundaries of the organization, ensuring better control, ownership, and security.

The Portable On-Premise IoT Cloud Mini Server enables IoT devices such as ESP32, Arduino, and Raspberry Pi to communicate using standard HTTP protocols, ensuring compatibility with existing development practices. By replacing remote cloud services with a local server, it provides secure authentication, real-time data monitoring, efficient time-series data storage, and multi-user project management. The system offers a low-cost, energy-efficient, secure, and scalable solution, making it ideal for IoT education, research, and small-scale industrial applications. Fig.1 clearly demonstrates how the proposed system integrates IoT devices, local server infrastructure, secure communication, and user interaction into a single, compact, and efficient platform.



The architecture of the proposed system is organized into four clearly defined layers, each responsible for a distinct set of functions. This layered design makes the system modular, easy to extend, and straightforward to understand for students at the undergraduate level.

This is the bottom layer of the architecture, consisting of all the physical sensor devices that generate data. These include ESP32 and Arduino microcontroller boards equipped with sensors for temperature, humidity, pressure, light intensity, soil moisture, motion detection, and other parameters. These devices are programmed to periodically send sensor readings to the mini server over HTTP. Each device is assigned to a specific project and uses a unique API key to authenticate its data transmissions. The modularity of this layer allows any sensor-equipped device to participate in the ecosystem, making the platform highly adaptable for diverse academic experiments.

The Raspberry Pi 5 serves as the central intelligence of the system. It runs Ubuntu Server 24.04 LTS, a stable and enterprise-grade Linux distribution that provides a solid foundation for server-side applications. The backend is built using Node.js with Express.js, which handles all incoming API requests, validates API keys, processes sensor data, and interacts with the database. The Node.js environment is highly efficient for handling multiple concurrent sensor connections, making it well-suited for real-time IoT applications. Nginx is deployed as a reverse proxy in front of the Node.js server, providing an additional layer of security by hiding the application server from direct external access.

All sensor data is stored in MongoDB configured with time-series collections — a specialized database format optimized for storing and querying data indexed by time. This configuration allows the system to efficiently handle large volumes of timestamped readings while maintaining query performance for historical analysis. The time-series structure also makes the data directly consumable by machine learning frameworks, enabling on-premise AI model training using locally collected sensor records.

6.4 Application and Communication Layer

The top layer provides the user-facing interfaces. A web-based dashboard built with responsive HTML and JavaScript allows administrators and students to create projects, register sensors, monitor live data streams, analyze historical trends, and manage API keys. Real-time data visualization is powered by WebSocket connections, which enable bidirectional communication between the server and the browser meaning the dashboard updates automatically the moment new sensor data arrives, without requiring the user to refresh the page.

Fig. 2 shows the data flow from IoT sensor devices to the web dashboard through the local mini server, including data collection, secure processing, storage, and real-time visualization.

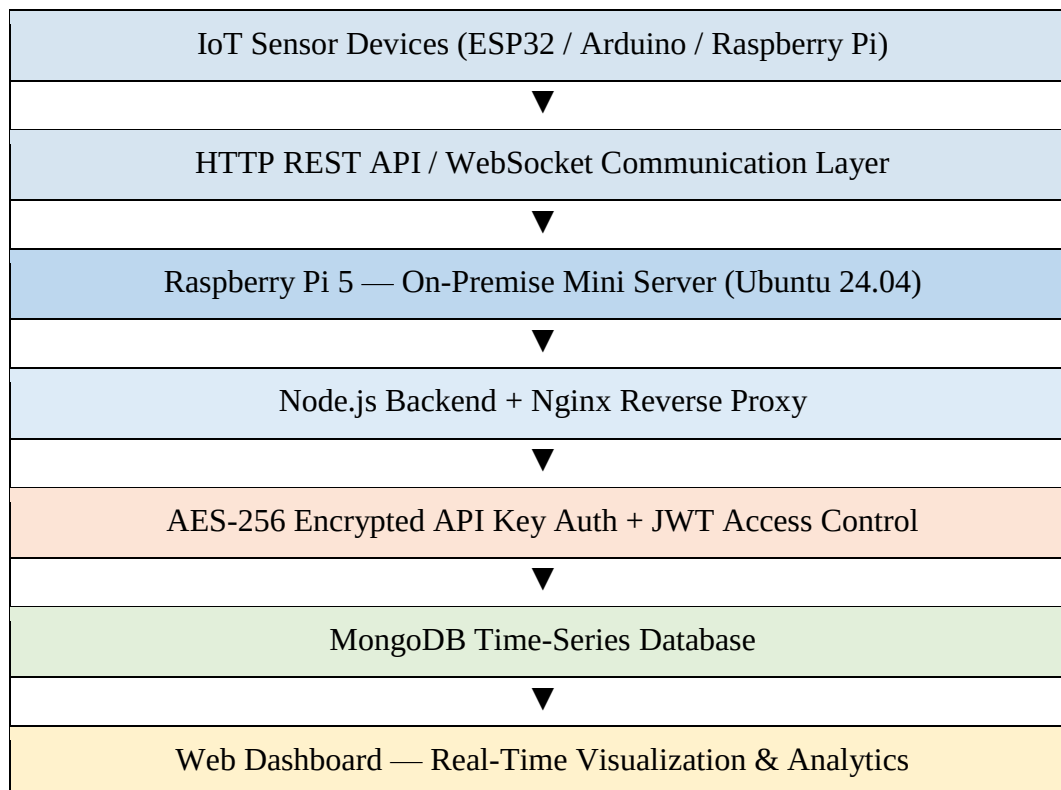


Fig. 2 - System Data Flow: From IoT Sensor Devices to Web Dashboard

7. Hardware and Software Implementation

The physical implementation of the system uses commercially available, affordable components that are accessible to most educational institutions. The following Table.2 provides a detailed overview of the hardware and software components used.

Table 2: System Components and Specifications

No.	Component	Description	Specification
1	Raspberry Pi 5	Core processing unit for backend and data management	Quad-core ARM64, 4–8 GB RAM
2	Ubuntu Server 24.04	Stable OS for server-side IoT services	Ubuntu LTS
3	Node.js + Express	REST API, authentication, data	Node.js v20 LTS

		ingestion logic	
4	MongoDB Time-Series	Structured storage for historical sensor data	Community Edition
5	WebSocket Engine	Real-time bidirectional data streaming	Socket.io Protocol
6	AES-256 Encryption	Encrypted API key storage and lifecycle management	GCM Mode
7	Argon2 + JWT	Password hashing and session authentication	Industry Standard
8	Nginx Reverse Proxy	Network isolation, HTTPS termination, rate limiting	Nginx Stable

8. Key Functional Modules

8.1 Secure User and Project Management

The system supports multiple users with role-based access control. An administrator can create and manage multiple IoT projects, add team members, and assign permission levels. Each project functions as an isolated workspace; sensor data from one project cannot be accessed by users of another project unless explicitly shared. This multi-project architecture mirrors the real-world structure of a university's IoT laboratory, where different student groups work on independent experiments simultaneously.

8.2 API Key-Based Data Ingestion

Every IoT project is assigned a unique API key that devices use to authenticate data transmissions. These keys are generated using cryptographically secure methods, stored in AES-256 encrypted form in the database, and validated on every incoming request. The system supports key rotation, meaning keys can be replaced without interrupting ongoing data collection and maintains an audit log of key usage events. This approach provides strong device-level security while remaining simple enough for undergraduate students to implement and understand.

8.3 Real-Time Data Processing and Streaming

When a sensor device sends data to the server, it is processed immediately by the Node.js backend. After validation and storage, the new data point is broadcast over a WebSocket channel to all connected dashboard clients in real time. Students and researchers watching the dashboard see live graphs update as sensor readings arrive, creating an engaging and educational real-time monitoring experience that mirrors professional IoT systems.

8.4 Historical Data Management

Stored sensor data can be queried over custom time ranges, filtered by sensor type, and exported for further analysis. MongoDB's time-series collection format ensures that these queries remain fast even as data volume grows over weeks and months of continuous operation. Students can download historical datasets directly from the dashboard for use in data science projects, statistical analysis, or machine learning training exercises.

8.5 Dynamic Sensor Auto-Registration

One of the most student-friendly features of the system is its dynamic sensor registration capability. When a device sends data containing a new parameter for example, a new field called 'soil_ph' the system automatically detects this unknown parameter and registers it as a new sensor type in the project's database schema. There is no need to manually pre-configure the database for every new sensor type. This dramatically reduces the setup friction for student projects and enables rapid prototyping with heterogeneous sensor arrays.

9. Security Framework

Security is a foundational requirement for any IoT platform, and the proposed system implements multiple overlapping security measures following a defense-in-depth philosophy. Each layer of the system contributes to overall data protection.

- **AES-256 Encryption:** All API keys stored in the database are encrypted using the Advanced Encryption Standard with 256-bit keys in GCM mode. Even if the database were compromised, API keys would remain unreadable without the encryption key.
- **JWT Authentication:** After login, users receive a JSON Web Token that must accompany every subsequent request. Tokens are time-limited and cryptographically signed, preventing unauthorized access and session hijacking.
- **Nginx Reverse Proxy:** The Node.js server is never directly exposed to the network. Nginx acts as an intermediary, handling HTTPS termination, filtering malformed requests, and providing rate limiting to protect against denial-of-service attacks.

Together, these mechanisms ensure that the system maintains data confidentiality, integrity, and availability — the three pillars of information security within the institutional network environment.

10. AI Integration for Future Classrooms

One of the most forward-looking aspects of the proposed system is its design as an AI-ready platform. As artificial intelligence increasingly intersects with IoT in industrial and educational environments, having locally available, well-structured sensor data becomes invaluable.

10.1 AI Readiness of the System

The system's MongoDB time-series storage format naturally produces structured, timestamped datasets that are directly compatible with popular machine learning libraries such as Python's scikit-learn, TensorFlow, and PyTorch. Students and researchers can query historical data from the dashboard, export it as CSV files, and immediately use it to train predictive models all without needing to move data to an external cloud platform.

10.2 AI Applications in Academic and MSME Contexts

The combination of real-time data streaming, structured historical storage, and a secure local compute environment opens the door to a wide range of AI-powered applications:

- **Predictive Maintenance:** In an MSME workshop, vibration or temperature sensors can feed a local anomaly detection model that alerts operators before equipment failure occurs.

- Smart Classroom Automation: Environmental sensors measuring CO₂ levels, temperature, and occupancy can drive an AI model that automatically adjusts ventilation and lighting for optimal learning conditions.
- Student Experiment Analytics: Sensor data from laboratory experiments can be automatically analyzed to detect outliers, verify expected physical relationships, and provide instant feedback to students.
- Agricultural IoT: Soil moisture, temperature, and pH sensors deployed in college farms can feed local predictive models for irrigation scheduling and yield optimization.

10.3 Role in Future Classrooms

The concept of a future classroom extends beyond smart whiteboards and video conferencing. It encompasses a learning environment where students interact with real physical systems, collect and analyze actual data, and build AI models grounded in real-world observations. The proposed IoT mini server serves as the central nervous system of such a classroom.

By providing a safe, fully controlled, and locally owned IoT infrastructure, institutions can encourage students to experiment freely without worrying about cloud billing overruns or accidental exposure of sensitive data. Faculty can design data-intensive IoT projects that span entire semesters knowing that all historical data remains accessible on the local server. Importantly, the hands-on experience of building, deploying, and securing this infrastructure gives students practical skills in server administration, API design, database management, and cybersecurity that are highly valued in the industry.

11. Performance and Feasibility Analysis

A key concern for any institution considering a locally deployed server is whether the system can realistically handle the demands of a busy academic environment. The performance characteristics of the proposed system have been carefully analyzed:

- Power Consumption: The Raspberry Pi 5 under active IoT workloads consumes less than 10 watts comparable to a smartphone charger.
- Concurrent Device Support: Node.js's event-driven, non-blocking I/O architecture allows the server to handle multiple simultaneous sensor connections efficiently.
- Storage Capacity: With a 128 GB or larger microSD card or an attached SSD, the server can store months or even years of high-frequency sensor data from a typical lab deployment.
- Network Performance: All communication happens within the campus LAN, which typically operates at 100 Mbps or higher — far exceeding the bandwidth requirements of even the most data-intensive IoT experiments.
- Cost Effectiveness: The total deployment cost of approximately ₹96,800 is a one-time investment that eliminates recurring cloud subscription fees.

12. Limitations and Future Scope

The proposed IoT Server provides secure local data storage, offline operation, and easy portability, making it suitable for educational and field applications. It offers hands-on learning in IoT server technologies, supports AI and machine learning through structured local data, and delivers a cost-effective alternative to commercial cloud platforms.

12.1 Current Limitations

While the system delivers significant value in its current form, certain limitations should be acknowledged:

- **Processing Power Boundaries:** The Raspberry Pi 5, while capable, has finite CPU and memory resources. Extremely high-frequency data collection from hundreds of simultaneous devices may approach the system's limits, requiring load balancing or hardware upgrades.
- **Single-Node Architecture:** The current design operates as a single server node. If the hardware fails, the system becomes unavailable until the node is restored. Hardware redundancy mechanisms are not yet implemented.
- **Local Access Only:** By design, the server is accessible only within the local network. Remote access requires additional configuration such as VPN setup, which adds complexity.

12.2 Future Scope

- **On-device AI:** Deploy lightweight AI models for real-time edge inference.
- **Federated Learning:** Enable collaborative model training without sharing raw data.
- **Multi-node Clustering:** Improve scalability and reliability using multiple Raspberry Pi servers.
- **Edge AI Acceleration:** Integrate AI accelerators for faster processing.
- **Digital Twin Integration:** Support predictive monitoring through virtual system models.

13. Conclusion

This chapter has presented the complete design, development, and analysis of a Portable On-Premise IoT Cloud Mini Server—a compact, secure, and AI-ready infrastructure platform specifically designed for future classrooms and MSME environments. The system addresses fundamental shortcomings of existing cloud-based IoT solutions: excessive cost, internet dependency, limited institutional control, and the absence of AI-ready local data pipelines.

Built on a Raspberry Pi 5 running Ubuntu Server 24.04 LTS, the platform integrates a Node.js backend with AES-256 encrypted API key management, Argon2 password hashing, JWT authentication, MongoDB time-series storage, WebSocket real-time streaming, and a responsive web dashboard delivering genuine cloud-like capabilities entirely within a local network.

Most significantly, the system's AI-ready architecture positions it as a foundation for the future classroom where IoT data collection, real-time analytics, and on-premise machine learning come together to create data-driven, intelligent, and engaging learning experiences. By bridging the gap

between IoT infrastructure, AI-driven learning, and secure data ownership, this work demonstrates that sophisticated IoT capabilities need not be confined to large organizations with large budgets they can and should be accessible to every learner and entrepreneur.

References

- [1]. Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2020). Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, 22(4), 2365–2420.
- [2]. Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2020). Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 7(8), 6733–6743.
- [3]. Xu, L. D., He, W., & Li, S. (2022). Internet of Things in industries: A survey. *IEEE Transactions on Industrial Informatics*, 18(3), 1495–1506.
- [4]. Zhang, Y., Deng, R. H., & Bertino, E. (2021). Security and privacy in smart IoT systems: Challenges and solutions. *IEEE Internet of Things Journal*, 8(1), 1–18.
- [5]. OWASP Foundation. (2023). OWASP API Security Top 10 — 2023. Retrieved from <https://owasp.org/API-Security>
- [6]. MongoDB Inc. (2022). Time-series data management in IoT applications. Retrieved from <https://www.mongodb.com/docs/manual/core/timeseries-collections>
- [7]. Raspberry Pi Foundation. (2024). Raspberry Pi 5 technical specifications. Retrieved from <https://www.raspberrypi.com/products/raspberry-pi-5>
- [8]. Antonopoulos, N., & Gillam, L. (Eds.). (2021). *Cloud computing: Principles, systems and applications* (2nd ed.). Springer.
- [9]. Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2019). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7), 1645–1660.
- [10]. Satyanarayanan, M. (2017). The emergence of edge computing. *Computer*, 50(1), 30–39.
- [11]. Murali, N & Murugan, S & Sivakumar, K & Vivekanandan, Manojkumar & Sushith, Mishmala & Sridharan, Manikandan. (2024). Smart Commodities Public Distribution System using IoT. *Salud, Ciencia y Tecnología - Serie de Conferencias*. 3. 624. 10.56294/sctconf2024624.
- [12]. Cisco Systems. (2022). Fog computing and the Internet of Things: Extend the cloud to where the things are. Cisco White Paper.
- [13]. Amazon Web Services. (2023). AWS IoT Core Developer Guide. Retrieved from <https://docs.aws.amazon.com/iot>
- [14]. Microsoft Azure. (2023). Azure IoT Hub Documentation. Retrieved from <https://learn.microsoft.com/en-us/azure/iot-hub>
- [15]. Google Cloud. (2022). Cloud IoT Core Documentation. Retrieved from <https://cloud.google.com/iot>
- [16]. Banks, A., & Gupta, R. (2014). MQTT version 3.1.1. OASIS Standard.
- [17]. N. Murali, R. Rajesh, S. Sridharan, A. E. P. Mariadas, S. P. Murugan and S. Manikandan, "A GPS-based Face Attendance Register System using Android Applications stored in the Cloud," 2024 11th International Conference on Computing for Sustainable Global Development (INDIACom), New Delhi, India, 2024, pp. 17-22, doi: 10.23919/INDIACom61295.2024.10498447.
- [18]. Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures. Doctoral dissertation, University of California, Irvine.
- [19]. Rescorla, E. (2018). The transport layer security (TLS) protocol version 1.3. IETF RFC 8446.
- [20]. Bonneau, J., Herley, C., van Oorschot, P. C., & Stajano, F. (2015). Passwords and the evolution of imperfect authentication. *Communications of the ACM*, 58(7), 78–87.
- [21]. Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107–113.
- [22]. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6), 84–90.